

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Deemed to be University, Bhubaneswar - 751024, Odisha

MID SEMESTER EXAMINATION (AUTUMN 2023) — SOLUTIONS & MARKING SCHEME

Subject: Data Structure (CS 2001)

Time: 1.5 Hours

Semester / Programme: 3rd Semester

Full Marks: 20

Instructions: Answer Question No. 1 (Compulsory) and any THREE from Question No. 2 to 5. Figures in brackets indicate marks.

SECTION - A (Compulsory)

1(a) If a 2D array $A[1..10][1..20]$ is stored in Row-Major order with base address 1000 and each element occupying 2 bytes, calculate the address of $A[5][10]$. [1]

$$\text{Address}(A[i][j]) = \text{Base} + w[N(i - L_1) + (j - L_2)]$$

Solution & Key Concept:

- Given: $\text{Base} = 1000$, $w = 2$, $L_1 = 1$, $L_2 = 1$, $N = 20$ (number of columns).
- Using Row-Major formula: $\text{Addr} = 1000 + 2 [20(5 - 1) + (10 - 1)]$.
- $\text{Addr} = 1000 + 2 [20(4) + 9] = 1000 + 2 [89] = 1000 + 178$.

Answer: 1178

Marking Rubric: [0.5 Mark] for correct formula application, [0.5 Mark] for final calculation.

1(b) Convert the following infix expression to postfix notation: $A + B * C / D$. [1]

Solution & Key Concept:

- Operator precedence: $*$ and $/$ are higher than $+$. They are left-associative.
- Process $B * C$: $BC*$.
- Process $(BC*) / D$: $BC*D/$.
- Process $A + (BC*D/)$: $ABC*D/+$.

Answer: $ABC*D/+$

Marking Rubric: [1 Mark] for correct postfix string.

1(c) How many null pointers are present in a Binary Tree with n nodes? [1]

$$\text{Null Pointers} = 2n - (n - 1)$$

Solution & Key Concept:

- Every node in a binary tree has 2 pointers, so total pointers = $2n$.
- Every node except the root has exactly one incoming edge (pointer), so used pointers = $n - 1$.
- Null pointers = $2n - (n - 1) = n + 1$.

Answer: $n + 1$

Marking Rubric: [1 Mark] for correct count/formula.

1(d) Define a Circular Queue and write the condition for a 'Queue Full' state in its array implementation of size N (using front and rear pointers). [1]

Solution & Key Concept:

- Definition: A circular queue is a linear data structure in which the last position is connected back to the first position to make a circle.
- Condition: $(\text{rear} + 1) \% N == \text{front}$.

Answer: $(\text{rear} + 1) \% N == \text{front}$

Marking Rubric: [0.5 Mark] for definition, [0.5 Mark] for correct condition.

1(e) What is the primary advantage of a Doubly Linked List (DLL) over a Singly Linked List (SLL)?

[1]

Solution & Key Concept:

- In SLL, we can only traverse in one direction (forward).
- In DLL, nodes have both 'next' and 'prev' pointers, allowing bidirectional traversal and easier deletion of a given node.

Answer: Bidirectional traversal and more efficient deletion of a known node.

Marking Rubric: [1 Mark] for mentioning bidirectional traversal or efficient deletion.

SECTION - B (Answer any THREE)

Answer any THREE questions from Q.2 to Q.5. Each question carries 5 marks.

Q.2. (a) Write a pseudocode or function to insert a node at the end of a Singly Linked List.

[2.5]

Solution for (a):

- 1. Create a new node and assign data.
- 2. If head is NULL, set head to new node.
- 3. Else, traverse the list until the last node (where $temp \rightarrow next == NULL$).
- 4. Update $last_node \rightarrow next = new_node$ and $new_node \rightarrow next = NULL$.

Result / Conclusion: void insertEnd(Node** head, int data) { ... }

Marking Rubric: [1 Mark] for handling empty list, [1.5 Marks] for correct traversal and pointer update.

(b) Represent the following sparse matrix in 3-tuple format: $\{bmatrix\} 0 \ \& \ 0 \ \& \ 5 \ \backslash \ 0 \ \& \ 8 \ \& \ 0 \ \backslash \ 3 \ \& \ 0 \ \& \ 0$
 $\{bmatrix\}$

[2.5]

Solution for (b):

- The 3-tuple format first row contains [Total Rows, Total Cols, Total Non-zero elements].
- Subsequent rows contain [Row Index, Col Index, Value].
- Non-zero elements: (0,2)=5, (1,1)=8, (2,0)=3.

Result / Conclusion: [[3, 3, 3], [0, 2, 5], [1, 1, 8], [2, 0, 3]]

Marking Rubric: [1 Mark] for header row, [1.5 Marks] for correct mapping of elements.

Q.3. (a) Evaluate the following postfix expression using a stack: 6, 2, 3, +, *, 8, 4, /, -. Show stack status at each step. [3]

Solution for (a):

- Push 6, Push 2, Push 3. Stack: [6, 2, 3]
- Op '+': pop 3, pop 2, push (2+3=5). Stack: [6, 5]
- Op '*': pop 5, pop 6, push (6*5=30). Stack: [30]
- Push 8, Push 4. Stack: [30, 8, 4]
- Op '/': pop 4, pop 8, push (8/4=2). Stack: [30, 2]
- Op '-': pop 2, pop 30, push (30-2=28). Stack: [28]

Result / Conclusion: 28

Marking Rubric: [2 Marks] for showing stack trace, [1 Mark] for final value.

(b) Explain the concept of Garbage Collection and Compaction in the context of dynamic storage management. [2]

Solution for (b):

- Garbage Collection: The process of automatically identifying and reclaiming memory that is no longer being used by the program (unreachable nodes).
- Compaction: The process of moving all allocated memory blocks to one end of the memory space to create a single large contiguous block of free memory.

Result / Conclusion: Garbage collection reclaims unused memory; compaction reduces fragmentation.

Marking Rubric: [1 Mark] for Garbage Collection, [1 Mark] for Compaction definition.

Q.4. (a) Construct a Binary Search Tree (BST) by inserting the following elements in sequence: 50, 30, 20, 40, 70, 60, 80. Draw the final tree. [3]

Solution for (a):

- Insert 50 (Root).
- Insert 30: $30 < 50$, goes left of 50.
- Insert 20: $20 < 50$, $20 < 30$, goes left of 30.
- Insert 40: $40 < 50$, $40 > 30$, goes right of 30.
- Insert 70: $70 > 50$, goes right of 50.
- Insert 60: $60 > 50$, $60 < 70$, goes left of 70.
- Insert 80: $80 > 50$, $80 > 70$, goes right of 70.

Result / Conclusion: Root(50) -> L(30), R(70); 30 -> L(20), R(40); 70 -> L(60), R(80)

Marking Rubric: [1.5 Marks] for correct logic of placement, [1.5 Marks] for final tree structure.

(b) Write a recursive function in C/C++ to find the height of a Binary Tree. [2]

$Height(T) = 1 + \max(Height(T.left), Height(T.right))$

Solution for (b):

- Base case: if node is NULL, return -1 (or 0 depending on definition).
- Recursive step: calculate height of left and right subtrees.
- Return 1 + maximum of the two heights.

Result / Conclusion: int height(Node* root) { if(!root) return -1; return 1 + max(height(root->left), height(root->right)); }

Marking Rubric: [1 Mark] for base case, [1 Mark] for recursive logic.

- Q.5. (a)** A binary tree has the following traversal sequences: In-order: D, B, E, A, F, C, G Pre-order: A, B, D, E, C, F, G Construct the unique binary tree. [3]

Solution for (a):

- Pre-order first element is root: A .
- In-order: Left of A is (D, B, E) , Right of A is (F, C, G) .
- In sub-list (D, B, E) , Pre-order says B is root. In-order says D is left, E is right.
- In sub-list (F, C, G) , Pre-order says C is root. In-order says F is left, G is right.

Result / Conclusion: Root: A ; Left: B (Left: D , Right: E); Right: C (Left: F , Right: G)

Marking Rubric: [1 Mark] for identifying root, [2 Marks] for correct left/right subtree construction.

- (b)** Explain how a polynomial like $P(x) = 4x^3 + 6x^2 + 10$ can be represented using a Linked List. [2]

Solution for (b):

- Each node in the linked list represents one term of the polynomial.
- A node structure should contain: Coefficient (int), Exponent (int), and Next Pointer.
- Example: $(4, 3, next) \rightarrow (6, 2, next) \rightarrow (10, 0, NULL)$.

Result / Conclusion: Linked list nodes store (coefficient, exponent) pairs linked in descending order of exponents.

Marking Rubric: [1 Mark] for node structure, [1 Mark] for example/diagram.

--- END OF PAPER ---